ChromaForge Comprehensive Documentation

1. Background & Novelty

Manual assembly of Sanger sequences is a time-consuming process highly susceptible to human error. Previous automation attempts, such as the Python script developed by Chen et al. (2021), have fundamental flaws: their algorithms rely heavily on rigid file naming conventions, require external software installation (EMBOSS), and fail to consider quality scores because they only process simple text formats (.seq).

ChromaForge is designed to bridge these technical gaps. The program operates independently without external dependencies. It extracts binary chromatogram data (.ab1) directly, trims raw data based on a Phred Q≥20 threshold, and applies Fuzzy Logic to pair Forward and Reverse sequences without requiring manual renaming intervention.

2. Key Features

- Native .ab1 Parsing: Reads chromatogram binary files directly without format conversion.
- Phred Q≥20 Auto-Trim: Automatically trims sequence ends based on mathematical error probabilities (Phred Q-scores).
- AI Smart Match (Fuzzy Logic): Intelligently pairs F and R files from a directory based on sample name similarities.
- ChromaCore AI Integration: Automated post-assembly evaluation using a large language model to diagnose sequence quality.
- Fault Tolerance: Isolates corrupted or problematic files without triggering a system crash.
- Interactive Dry-Run Module: Provides a preview interface of sample pairing drafts with flexible commands (tukar, geser, pisah) prior to assembly.
- Gap-Aware Consensus Matrix: Intelligently accommodates indels, ensuring high-quality peak bases are retained instead of being dropped into empty spaces or 'N'.

3. Detailed Logic Flow

3.1. Automated Sample Pairing (AI Smart Match / Fuzzy Logic & Dry-Run)

The script scans the designated directory for .ab1 files. Using string matching and core IDs, it pairs Forward (F) and Reverse (R) reads strictly and universally to prevent cross-contamination. Unpaired files are retained for single-read processing.
Before assembly execution, the interactive Dry-Run module displays the sample pairing draft in the terminal and allows manual adjustments using commands like tukar, geser, or pisah.

3.2. Robust Quality Trimming & Masking

The robust_trim_masking_dan_evaluasi function is applied to every read before alignment:

- End Trimming: The algorithm uses a sliding window of 5 bases. It scans from the 5' end until it finds a window where the average Phred Q-score is ≥ 20 to set the start index. It then scans from the 3' end to find the first window (moving backwards) averaging ≥ 20 to set the end index. Bases outside these boundaries are discarded.
- Internal Masking: Within the trimmed sequence, any individual base with a Q-score < 20 is masked (replaced) with an 'N'.
- Quality Scoring: The script calculates a fitness score based on the ratio of good bases to the total length of the trimmed sequence.

3.3. Length & Quality Threshold Evaluation

After trimming, two strict gates are applied:

- Minimum Length Gate: Any trimmed sequence shorter than 200 bp is instantly discarded (SKIPPED).
- Assembly Gate: The script requires a fitness score of ≥ 85% for both F and R sequences to proceed with pairwise contig assembly.

3.4. Alignment, Gap-Aware Consensus, and IUPAC Handling

If both reads pass the gates, the R read is reverse-complemented (Rc). A local alignment is performed using Biopython's PairwiseAligner with a gap-aware consensus matrix:

- Identical Bases: If F and Rc match, the base is added to the consensus.
- Mismatch Resolution (Delta > 10): The base with a Q-score greater by more than 10 points wins and overwrites the weaker base.
- Double Heterozygote Resolution (Delta ≤ 10, Both Q ≥ 30): If both signals are strong but conflicting, this indicates true polymorphism. The script prints an IUPAC ambiguity base (e.g., R, Y, S) specifically for the DnaSP output file. Simultaneously, it performs pseudo-phasing by selecting the marginally stronger base for the MEGA output file.
- Noise (Delta ≤ 10, Q < 30): The mismatch is considered noise, and an 'N' is assigned.

3.5. Orientation Correction (NCBI BLASTn)

If enabled, finalized contigs are queried against NCBI BLASTn ('nt' database). The script evaluates the HSP frame; if a minus frame is detected, it automatically reverse-complements the final sequence.

3.6. Output Files

Depending on IUPAC detection and API availability, the script generates dual output files ensuring no file overwriting occurs:

- Individual Files per Sample: Saved with custom prefixes (e.g., [CustomName]_[ID]_Final.fasta / For_DnaSP.fasta / For_MEGA.fasta).
- MultiFASTA Bundles: MultiFASTA_Final.fasta, MultiFASTA_For_MEGA.fasta, and MultiFASTA_For_DnaSP.fasta.
- Mismatch_Alignment_Report.txt: A technical log recording every mismatch point.
- AI_Analysis_Result.txt: A narrative evaluation from ChromaCore AI classifying sample quality along with lab reminders.

Step 1: Workspace Preparation

1. Create a new folder on your PC (e.g., 'DNA_Project').
2. Copy all your .ab1 files into this folder.
3. Place the Python script (ChromaForge_Assembler.py) in the same folder.

Step 2: ChromaCore AI Configuration (Optional)

To activate the automated evaluation assistant (ChromaCore AI), follow these specific instructions to avoid common .txt file extension issues:

1. You must generate your own API Key via Google AI Studio (https://aistudio.google.com/). Do not use or share other people's keys.

2. Open the Notepad application on your computer.

3. Type the following code on the first line (replace the text with your actual key, without quotes):

`GEMINI_API_KEY=your_secret_key_here`

4. Click File > Save As.

5. CRITICAL: In the 'Save as type' dropdown (below the filename), change 'Text Documents (*.txt)' to 'All Files (*.*)'.

6. In the 'File name' field, type exactly .env (ensure there is a dot at the beginning).

7. Save it inside the 'DNA_Project' folder (the same folder as the script). If the file icon still looks like a regular Notepad file named '.env.txt', the CMD system will fail to read it. Ensure the file type is listed as 'ENV File'.

Step 3: System Requirements Installation

A. Install Python

1. Download Python from https://www.python.org/downloads/
2. Run the installer. CRITICAL: Check the box that says 'Add Python to PATH' at the bottom before clicking 'Install Now'.

B. Install Biopython (For DNA processing)

1. Open Command Prompt (CMD) from the Windows menu.
2. Type the following command and press Enter:

```
pip install biopython
```

C. Install Gemini AI Libraries (For ChromaCore AI)

1. In the same CMD window, type the following command and press Enter:

```
pip install google-genai python-dotenv
```

2. Wait for the installation to complete. This is required for the script to connect to Google's AI services.

D. Updating Libraries (For Future Use)

If you need to update the libraries to their latest versions in the future, open CMD, type the following command, and press Enter:

```
pip install --upgrade google-genai biopython
python-dotenv
```

Step 4: Running the Script

1. Open the 'DNA_Project' folder. Click the Address Bar at the top, delete the text, type cmd, and press Enter.
2. In CMD, type: **`python ChromaForge_Assembler.py`** and press Enter.
3. When prompted for the folder name, just press Enter if your files are in the same directory.
4. Select BLASTn Mode (Type 'n' for instant Offline processing, or 'y' for Online NCBI verification).
5. Follow the interactive Dry-Run prompts, make manual adjustments if needed, and input your custom output name.
6. Monitor the terminal progress. If the AI and .env configuration is correct, the terminal will display 'Status ChromaCore AI: AKTIF'. The final outputs will automatically save in your folder.